

Acceso a Datos - Semana 2

UD1. Manejo de ficheros - Sistemas de ficheros

Objetivo de la semana: aprender a localizar, crear, consultar, recorrer, copiar, mover y eliminar ficheros y directorios desde Java.

1. ¿Qué es un sistema de ficheros?

El sistema de ficheros es la forma en la que el sistema operativo organiza y almacena la información en dispositivos como discos duros, SSD o memorias USB. La información se estructura mediante ficheros y directorios.

- Fichero: conjunto de datos almacenados con un nombre.
- Directorio o carpeta: contenedor que agrupa ficheros y otros directorios.
- Ruta: cadena que indica la localización de un fichero o directorio.

2. Rutas absolutas y relativas

Ruta absoluta: parte desde la raíz del sistema y localiza el recurso de manera completa.

```
C:\Users\alumno\Documents\datos.txt
```

Ruta relativa: se interpreta a partir del directorio de trabajo actual de la aplicación.

```
datos\alumnos.txt
```

Las rutas relativas suelen ser preferibles en proyectos porque hacen que el programa sea más portable entre equipos.

3. Separadores de ruta y portabilidad

Cuando accedemos a los sistemas de ficheros, tenemos que plantearnos en qué sistema operativo está ejecutándose nuestro programa. En todos los sistemas operativos actuales existe la gestión en directorios y ficheros, pero debemos tener en cuenta el separador, referido al carácter que separa un elemento contenido dentro de otro cuando especificamos las rutas de carpetas. En sistemas basados en Linux es la barra (slash, /) mientras que en los sistemas Microsoft es la contrabarra (backslash, \).

Para que nuestros programas sean compatibles con ambos sistemas debemos utilizarla constante `file.separator`. Debemos tener especial atención en los sistemas operativos de Microsoft ya que la contra barra tiene un significado especial dentro de los strings cosa que no ocurre con la barra de los sistemas Linux.

```
File fichero=new File("/home/victor/prueba.md") //sistemas operativos Linux
```

```
File fichero=new File("C:\\Usuarios\\Victor\\Escritorio\\prueba.md") //sistemas operativos Windows
```

Windows utiliza normalmente la barra invertida (\) y Linux/macOS la barra normal (/). Java ofrece mecanismos para construir rutas sin depender de la plataforma.

```
Path ruta = Path.of("datos", "alumnos.txt");
```

4. La clase File

La clase `java.io.File` representa una ruta a un fichero o directorio. No contiene directamente el contenido del fichero; permite consultar y manipular propiedades del sistema de ficheros.

Los constructores principales que posee son:

<code>File (File parent, String child)</code>	Crea el acceso a un elemento (child) a partir de una objeto file existente (Parent) que representa a un directorio punto el constructor está sobrecargado coma y el primer elemento puede ser simplemente un string.
<code>File(String pathname)</code>	Crea el acceso a un elemento mediante una ruta absoluta, o bien un elemento dentro de la ruta de ejecución.

```
File fichero = new File("datos.txt");

System.out.println(fichero.exists());
System.out.println(fichero.getName());
System.out.println(fichero.getAbsolutePath());
System.out.println(fichero.length());
System.out.println(fichero.isFile());
System.out.println(fichero.isDirectory());
```

Métodos habituales de File

Método/Constructor	Función/Descripción
<code>File (String pathname)</code>	Crea un nuevo objeto de tipo File a partir de su ruta
<code>Boolean createNewFile ()</code>	Crea un nuevo archivo vacío con la ruta definida por el File
<code>Boolean delete()</code>	Elimina el fichero o directorio si es posible.
<code>Boolean exists()</code>	Indica si el fichero o directorio existe.
<code>Boolean isFile()</code>	Indica si la ruta corresponde a un fichero.
<code>Boolean isDirectory()</code>	Indica si corresponde a un directorio.
<code>String getName()</code>	Devuelve el nombre del fichero/directorio sin la ruta.
<code>getAbsolutePath()</code>	Devuelve la ruta absoluta.
<code>length()</code>	Devuelve el tamaño en bytes.
<code>mkdir()</code>	Crea un directorio.
<code>mkdirs()</code>	Crea el directorio y los padres necesarios.
<code>File listFiles()</code>	Devuelve el contenido de un directorio.

Si queremos obtener más información, podemos ir a la API de Java, tecleando `api java 20`, vamos al `java.base`, luego al `java.io` para profundizar en la clase File.

5. Crear directorios con File

```
File carpeta = new File("datos");

if (!carpeta.exists()) {
    boolean creada = carpeta.mkdir();
    System.out.println("Carpeta creada: " + creada);
}
```

`mkdir()` crea un único directorio. `mkdirs()` permite crear también los directorios padre que falten.

6. Recorrer el contenido de una carpeta

```
File carpeta = new File("datos");
File[] elementos = carpeta.listFiles();

if (elementos != null) {
    for (File elemento : elementos) {
        if (elemento.isDirectory()) {
            System.out.println("[DIR] " + elemento.getName());
        } else {
            System.out.println("[FILE] " + elemento.getName());
        }
    }
}
```

7. Path y Files: la API moderna de Java

Desde Java 7 existe la API NIO.2. Las clases Path y Files ofrecen una forma más moderna, clara y potente de trabajar con el sistema de ficheros.

```
Path ruta = Path.of("datos", "alumnos.txt");

System.out.println(ruta);
System.out.println(ruta.toAbsolutePath());
System.out.println(ruta.getFileName());
System.out.println(ruta.getParent());
```

8. Operaciones frecuentes con Files

```
Path carpeta = Path.of("datos");
Path fichero = Path.of("datos", "alumnos.txt");

Files.createDirectories(carpeta);

if (!Files.exists(fichero)) {
    Files.createFile(fichero);
}

System.out.println(Files.exists(fichero));
System.out.println(Files.size(fichero));
```

9. Copiar, mover y borrar

```
Path origen = Path.of("datos", "alumnos.txt");
Path copia = Path.of("copias", "alumnos.txt");

Files.createDirectories(copia.getParent());
Files.copy(origen, copia);
Path destino = Path.of("datos", "alumnos_2026.txt");
Files.move(origen, destino);
Files.delete(destino);
```

Estas operaciones pueden lanzar excepciones, por lo que normalmente se utilizan dentro de un bloque try-catch o en métodos que declaran throws IOException.

10. Excepciones de entrada/salida

Trabajar con ficheros implica situaciones que el programa no controla completamente: permisos insuficientes, fichero inexistente, dispositivo lleno, ruta incorrecta, etc. Muchas operaciones de Files pueden lanzar IOException. Por ello debemos incluirlo dentro de un bloque try catch o propagar esa excepción.

```
try {
    Path ruta = Path.of("datos", "prueba.txt");
    Files.createFile(ruta);
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

11. File frente a Path/Files

File	Path / Files
API clásica de java.io	API moderna NIO.2
Operaciones básicas	Más operaciones y mejor manejo de errores
Representa fichero/directorio	Path representa rutas; Files realiza operaciones
Sigue siendo válida	Preferible para código nuevo

12. Qué debes saber al finalizar esta semana

- Distinguir fichero, directorio y ruta.
- Diferenciar rutas absolutas y relativas.
- Consultar información de ficheros y carpetas.
- Crear directorios y ficheros.
- Recorrer el contenido de una carpeta.
- Copiar, mover y eliminar recursos.
- Comprender la diferencia entre File y Path/Files.
- Reconocer y gestionar errores básicos de entrada/salida.

Ejemplos de File

```
// ejemplo fichero
File fichero = new File("ejemplo1.txt");
if(fichero.exists()) System.out.println("El fichero existe");
else System.out.println("El fichero no existe");
System.out.println("Nombre: " + fichero.getName());
System.out.println("Longitud: " + fichero.length());
System.out.println("Ruta absoluta: " + fichero.getAbsolutePath());

// ejemplo carpeta
File carpeta = new File("ruta_carpeta");
if(carpeta.exists()) System.out.println("La carpeta existe");
else System.out.println("La carpeta no existe");
System.out.println("Nombre: " + carpeta.getName());
System.out.println("Longitud: " + carpeta.length());
System.out.println("Ruta absoluta: " + carpeta.getAbsolutePath());
```

Ejemplos de rutas con File:

```
File carpetaActual = new File("."); // carpeta actual
File carpetaPadre = new File(".."); // carpeta superior → C:/Users/VIRUS/Desktop
File carpetaRaiz = new File("C:/"); // carpeta raíz de la unidad C: en Windows
File carpeta1 = new File("C:/Users/VIRUS"); // carpeta VIRUS unidad C: en Windows
File carpeta2 = new File("../"); // carpeta VIRUS unidad C: en Windows
File carpeta3 = new File("../.."); // carpeta Users unidad C: en Windows
File carpeta4 = new File("../imgs"); // C:/Users/VIRUS/Desktop/imgs

File archivo1 = new File("../img1.png"); // C:/Users/VIRUS/Desktop/img1.png
File archivo2 = new File("C:/img2.png"); // C:/img2.png
File archivo3 = new File("../..img3.png"); // C:/Users/VIRUS/img3.png
File archivo4 = new File("img4.png"); // carpeta actual → img4.png
File archivo5 = new File("imgs/img5.png");
// C:/Users/VIRUS/Desktop/curso-programacion-java/imgs/img5.png
```

Ejemplos de Crear y Borrar Archivos y directorios

```
File archivo = new File("nuevoArchivo.txt");
try {
    if (archivo.createNewFile()) System.out.println("Archivo creado");
    else System.out.println("El archivo ya existe");
} catch (IOException e) { e.printStackTrace(); }

File directorio = new File("nuevoDirectorio");
if (directorio.mkdir()) System.out.println("Directorio creado");
else System.out.println("No se pudo crear el directorio");

File archivoBorrar = new File("archivoParaEliminar.txt");
if (archivoBorrar.delete()) System.out.println("Archivo eliminado");
else System.out.println("No se pudo eliminar el archivo");

File directorio2 = new File(".");
String[] archivos = directorio2.list();
if (archivos != null) for (String a : archivos) System.out.println(a);
else System.out.println("No se pudo listar los archivos");
```

Ejemplo de uso de la clase File: listar ficheros de una carpeta.

```
public static void main(String[] args) {
    String rutaCarpeta = ".";
    listarArchivos(rutaCarpeta);
}

// método que lista todos los archivos de un carpeta, indicando el tamaño en bytes
public static void listarArchivos(String rutaCarpeta) {
    File carpeta = new File(rutaCarpeta);

    if (carpeta.isDirectory()) {
        File[] archivos = carpeta.listFiles();

        for (File f : archivos) {
            if (f.isFile()) {
                System.out.println(f.getName() + " - " + f.length() + "bytes");
            }
        }
    } else System.err.println("La ruta proporcionada no es una carpeta");
}
```

Ejemplo de uso de la clase File: lista todos los archivos de una carpeta y subcarpetas con su ruta:

```
public static void main(String[] args) {  
    String rutaCarpeta = ".";  
    listarArchivos(new File(rutaCarpeta));  
}  
  
// método que lista todos los archivos de un carpeta y sus subcarpetas, con su ruta  
// desde la ubicación actual. No deben aparecer las carpetas  
public static void listarArchivos(File carpeta) {  
    if (carpeta.isDirectory()) {  
        File[] archivos = carpeta.listFiles();  
  
        for (File archivo : archivos) {  
            if (archivo.isFile()) System.out.println(archivo.getPath());  
            else if (archivo.isDirectory()) listarArchivos(archivo);  
        }  
    }  
}
```